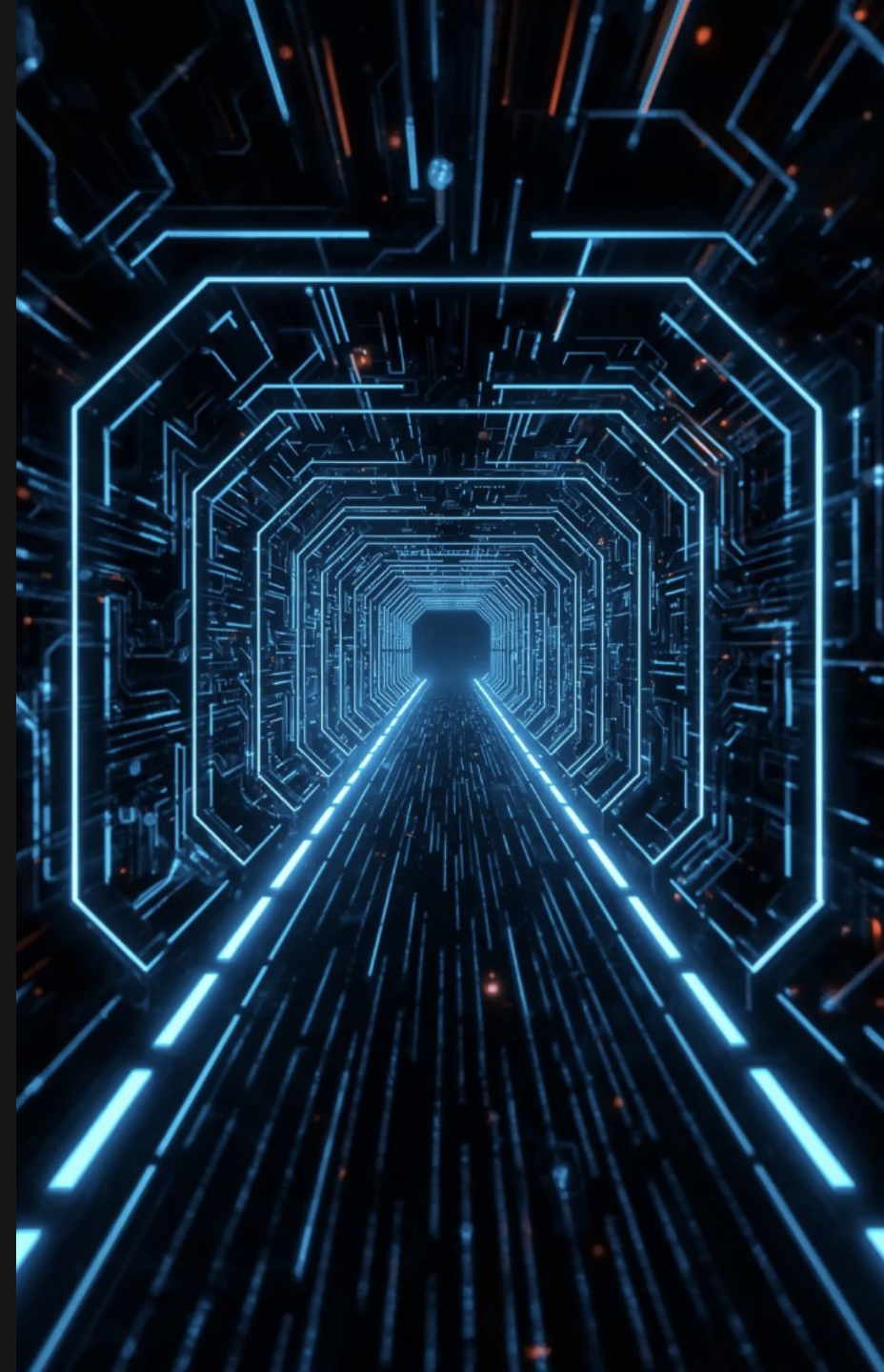


TITAN OPERATIONS SENTINEL

A Multi-Agent AI System for Manufacturing

Agentic AI for IT — Group Assignment | IE University, June 2026

Team: Marco, David, Ignacio, Nuria & Marian



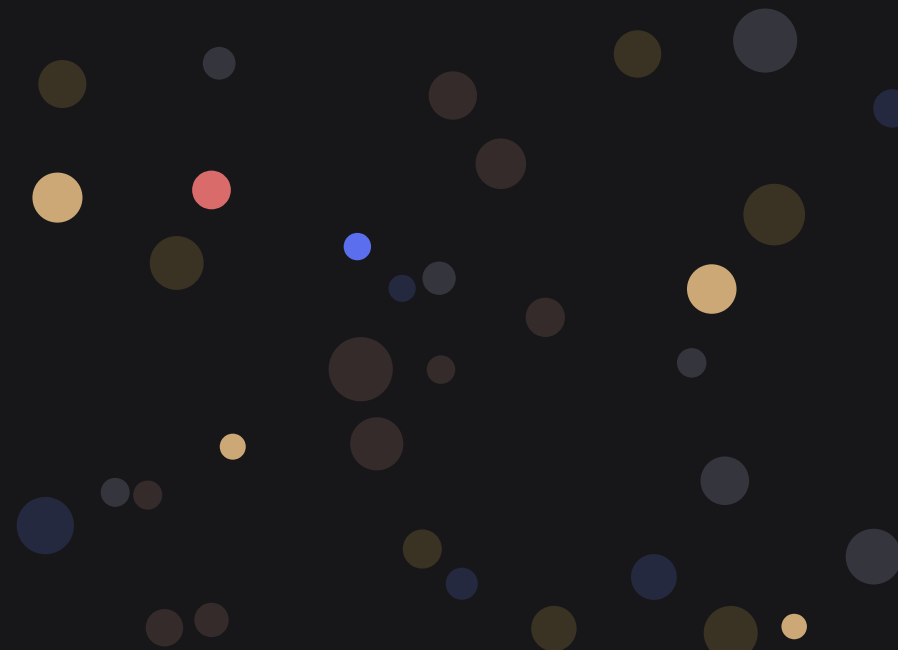
THE MANUFACTURING CHALLENGE

28 Plants. 22,000 Alerts a Day. No Way to Act.

Maintenance, supply chain, production, and safety teams work in silos.

Dashboards show data but don't decide; scripts can't reason across teams.

That's the gap an agent closes: it perceives the alert, reasons across every domain, and proposes a costed plan — a dashboard can't decide, and a script can't adapt.



28

plants worldwide



22,000+

sensor alerts / day, unprioritized



\$180K

lost per day per CNC machine
down



\$14M

lost last quarter to missing parts

BACKGROUND

Titan Manufacturing Corporation, Today

A global leader in industrial machinery, robotics components, and precision equipment — mixing advanced robotics with aging operational technology on every plant floor.

Symptoms leadership is watching:

- Unplanned downtime on critical CNC assets
- Supply chain volatility beyond Tier-1 visibility
- Plant-floor decision bottlenecks across five disconnected teams

Five teams. Zero shared visibility.

Reliability



Supply Chain



Production



Quality



56,000

EMPLOYEES



14

COUNTRIES



28

MANUFACTURING PLANTS

Five Teams, One Phone Tree, Five-Plus Hours to a Decision



5+ hours

from failure noticed to a coordinated action plan



< 5 minutes, with TOS

target time to a costed, safety-gated plan

Why not just automate the steps? Each handoff needs judgment a rule engine can't make — whether parts are worth expediting, which machine absorbs the rerouted jobs, whether a safety sign-off applies. That requires an agent that reasons across domains, not a longer script.

One Root Cause, Five Fault Lines — and a TOS Agent Assigned to Each

Production throughput is down 9% YoY. SLAs are breached by delayed materials, machine failures, and operator-robot conflicts.

Root cause: fragmented operational intelligence, isolated systems, entirely manual decision loops.

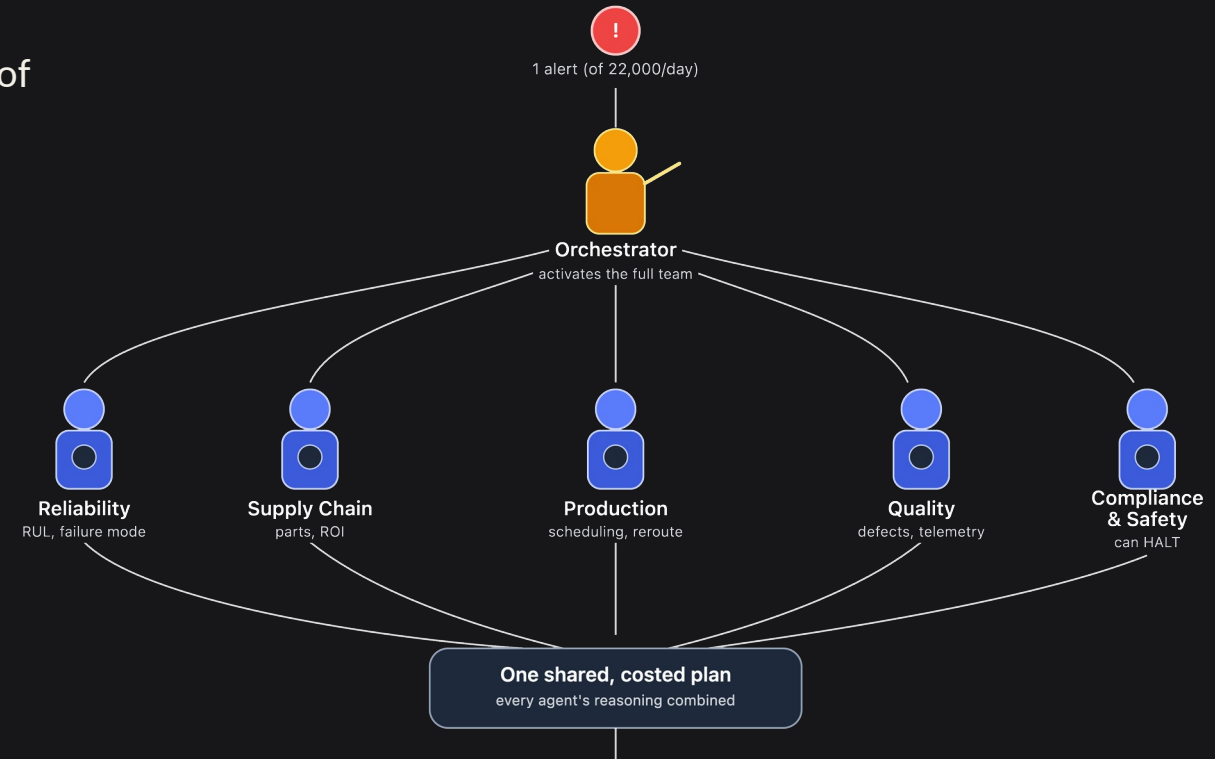
 Asset Reliability & Predictive Maintenance • 22,000+ unprioritized alerts/day • MTBF down 11% YoY • \$180K/day per CNC machine down RELIABILITY AGENT	 Supply Chain Volatility & Risk • Supplier delays up 28% • \$14M lost last quarter to missing parts • Tier-2 risk found only after a stall SUPPLY CHAIN AGENT	 Human-Robot Coordination • 17% of shifts hit scheduling conflicts • Safety shutdowns up 15% YoY • Robot cells can't adapt to staffing PRODUCTION AGENT	 Quality & Traceability • Quality escapes up 22% YTD • Root-cause analysis takes weeks • Relies on tribal knowledge QUALITY AGENT	 Compliance, Safety & Audit • OSHA gaps from incomplete logs • Audits cross-reference 20+ systems • Slow incident reconstruction COMPLIANCE & SAFETY AGENT
---	--	--	--	---

One Alert Wakes Up a Team of 6 AI Agents

TOS deploys **six specialized AI agents**, each owning one important part of the operation: reliability, supply chain, production scheduling, quality assurance, and safety.

When a single alert arrives, the **orchestrator activates** the full team.

Every agent contributes to the plan **independently**, uses its **own tools**, which is then sent to a human for final approval (**HITL**)



Five fault lines. One orchestrator. One shared, costed plan.

Applying the Agent Loop

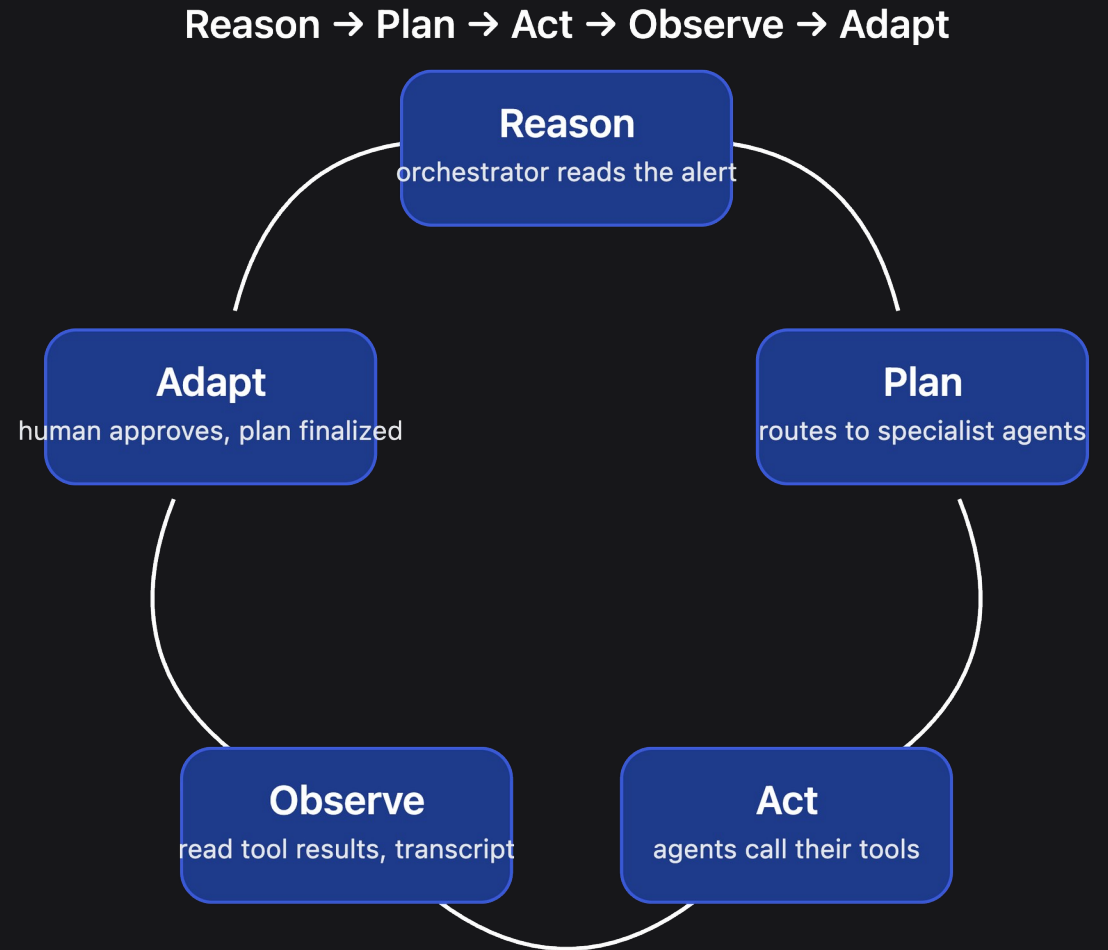
We used the **Reason → Plan → Act → Observe → Adapt** loop and embedded it into every agent.

Each incoming alert moves through:

1. Perceive the signal
2. Reason across specialists
3. Act via tools
4. Observe outcomes
5. Adapt to human approval

The loop closes back from Adapt to Reason, ensuring continuous refinement until a coordinated plan is approved.

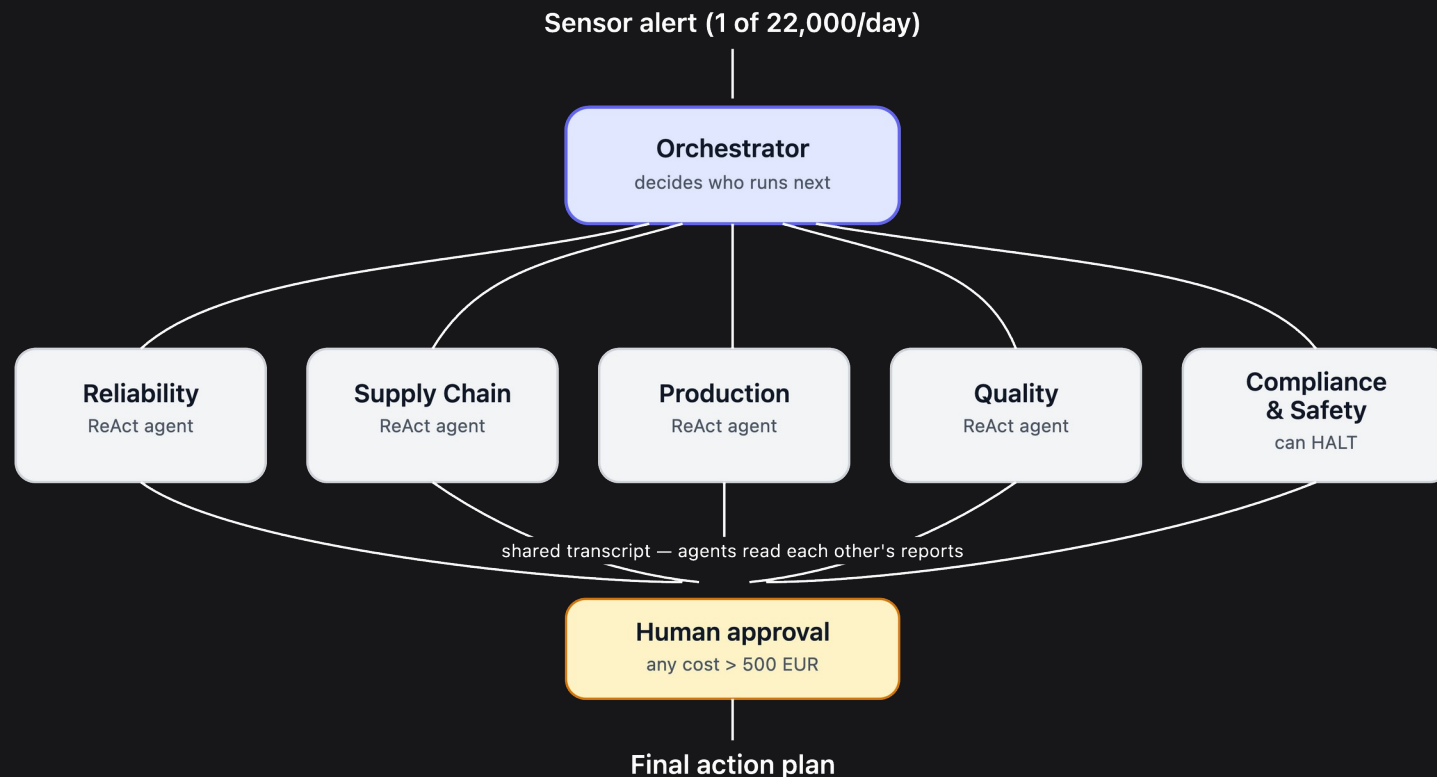
Five steps, one continuous loop — not a waterfall handoff.



Loops back from Adapt to Reason whenever more work is needed

Architecture: Orchestrator + ReAct Agents

- We chose the **Orchestrator/Supervisor** pattern: one router, five specialists, clear accountability.
- Each **specialist runs as a ReAct agent** that reasons, selects its own tools, and loops until done.
- **Why multi-agent?** 18 tools are too many decisions for one agent to carry well.
- **Built with LangGraph + LangChain:** a LangGraph StateGraph orchestrates five LangChain create_react_agent specialists.



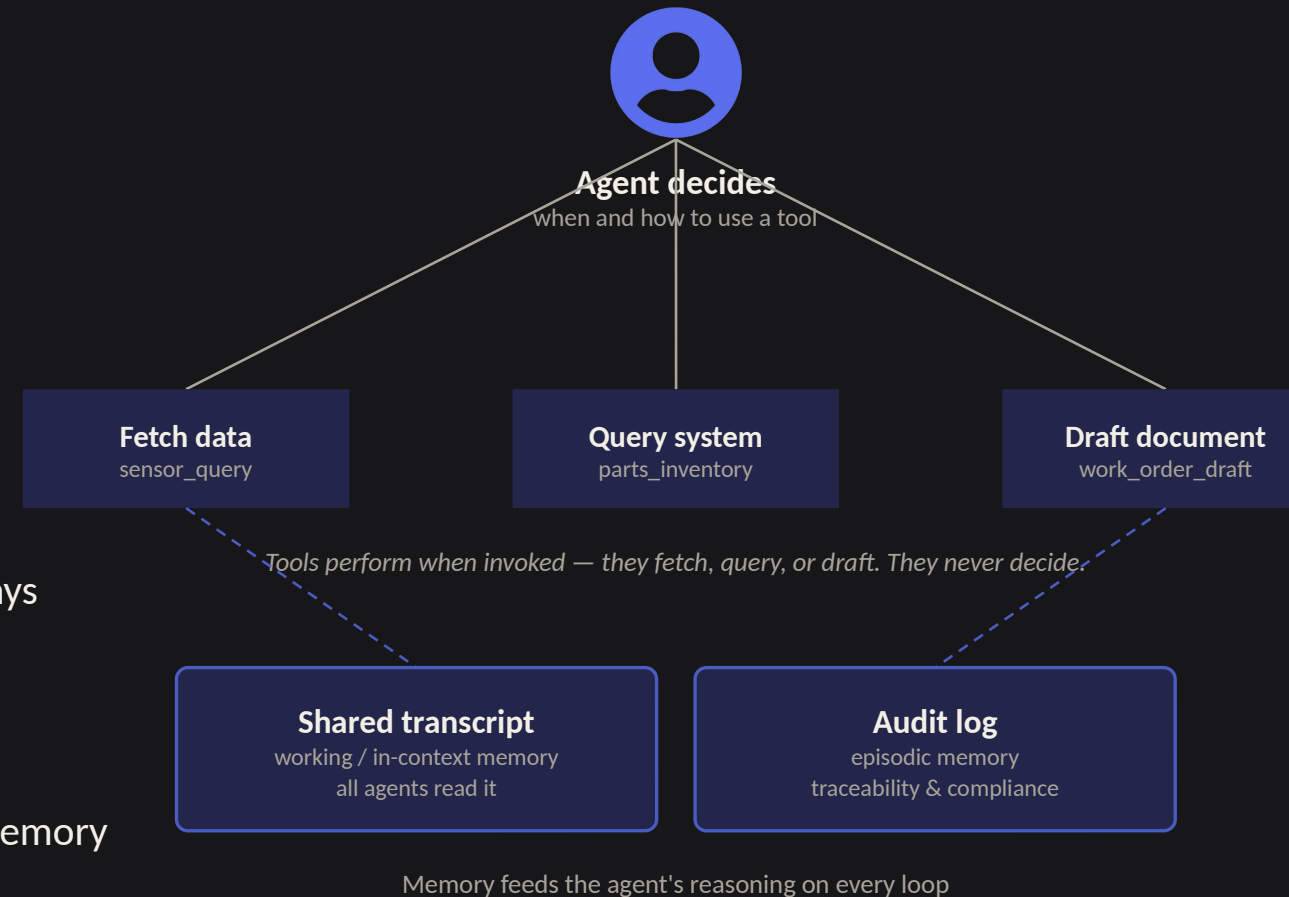
Tools Act, Agents Decide

A **tool** performs an operation when invoked: it fetches data, drafts a document, or triggers a system call.

An agent **decides when and how to use it**, looping until the task is resolved.

TOS's 18 **tools only fetch data or draft** documents; every decision stays with the agent who called them.

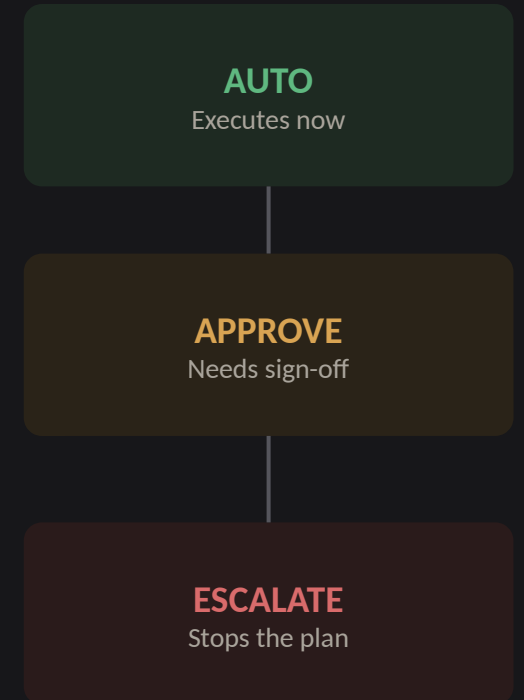
Memory at work: the shared transcript acts as **in-context** working memory across all agents; the audit log serves as **episodic memory**, enabling traceability and safety compliance.



Guardrails: 3 Levels of Agent Authority

- ✓ **AUTO:** Routine diagnostic and monitoring actions execute immediately (no human approval)
- ⚠ **APPROVE:** Any spend above threshold or schedule change is paused and routed to a human operator for sign-off before execution.
- 🔒 **ESCALATE / HALT:** A detected safety risk stops the entire plan instantly, no exceptions, no overrides.

ESCALATING AUTHORITY



What Could Go Wrong, and How We Catch It

Built-In Safeguards at Every Failure Point

-  **Low-confidence prediction → escalate, don't guess: TOS flags uncertainty and routes to human review rather than acting on weak signals.**
-  **Stale sensor data → flagged before use: every data input is timestamped and validated; outdated readings are quarantined before they reach the reasoning layer.**
-  **Spending ceiling enforced in code, not just the prompt: budget limits are hard-coded into tool logic, no prompt injection or agent drift can override them.**

Source: Session 5

3 failure points. 0 blind spots.
Every one caught before it reaches a decision.

